JUKEDRUMMER: CONDITIONAL BEAT-AWARE AUDIO-DOMAIN DRUM ACCOMPANIMENT GENERATION VIA TRANSFORMER VQ-VAE

Yueh-Kao Wu
Academia Sinica
yk.lego09@gmail.com

Ching-Yu Chiu
National Cheng Kung University
x2009971@gmail.com

Yi-Hsuan Yang
Taiwan AI Labs
yhyang@ailabs.tw

ABSTRACT

This paper proposes a model that generates a drum track in the audio domain to play along to a user-provided drum-free recording. Specifically, using paired data of drumless tracks and the corresponding human-made drum tracks, we train a Transformer model to improvise the drum part of an unseen drumless recording. We combine two approaches to encode the input audio. First, we train a vector-quantized variational autoencoder (VQ-VAE) to represent the input audio with discrete codes, which can then be readily used in a Transformer. Second, using an audio-domain beat tracking model, we compute beat-related features of the input audio and use them as embeddings in the Transformer. Instead of generating the drum track directly as waveforms, we use a separate VQ-VAE to encode the mel-spectrogram of a drum track into another set of discrete codes, and train the Transformer to predict the sequence of drum-related discrete codes. The output codes are then converted to a mel-spectrogram with a decoder, and then to the waveform with a vocoder. We report both objective and subjective evaluations of variants of the proposed model, demonstrating that the model with beat information generates drum accompaniment that is rhythmically and stylistically consistent with the input audio.

1. INTRODUCTION

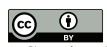
Deep generative models for musical audio generation have witnessed great progress in recent years [1–8]. While models for generating symbolic music such as MIDI [9–12] or musical scores [13] focus primarily on the *composition* of musical content, an audio-domain music generation model deals with *sounds* and thereby has extra complexities related to timbre and audio quality. For example, while a model for generating symbolic guitar tabs can simply consider a guitar tab as a sequence of notes [11], a model that generates audio recordings of guitar needs to determine not only the underlying sequence of notes but also the way to render (synthesize) the notes into sounds. Due to the complexities involved, research on deep generative models for musical audio begins with the simpler task of synthesizing individual musical notes [1–3], dispensing the need to consider the composition of notes. Follow-up research [4–6] extends the capability to generating musical passages of a single instrument. The Jukebox model [7] proposed by OpenAI greatly advances the state-of-the-art by being able to, quoting their sentence, "generate high-fidelity and diverse songs with coherence up to multiple minutes." Being trained on a massive collection of audio recordings with the corresponding lyrics but not the symbolic transcriptions of music, Jukebox generates multi-instrument music as raw waveforms directly without an explicit model of the underlying sequence of notes.

This work aims to improve upon Jukebox in two aspects. First, the backbone of Jukebox is a hundred-layer Transformer [14, 15] with billions of parameters that are trained with 1.2 million songs on hundreds of NVIDIA V100 GPUs for weeks at OpenAI, which is hard to reproduce elsewhere. Inspired by a recent Jukebox-like model for singing voice generation called KaraSinger [8], we instead build a light-weight model with only 25 million parameters by working on Mel-spectrograms instead of raw waveforms. Our model is trained with only 457 recordings on a single GeForce GTX 1080 Ti GPU for 2 days.

Second, and more importantly, instead of a fully autonomous model that makes a song from scratch with various instruments, we aim to build a model that can work cooperatively with human, allowing the human partner to come up with the musical audio of *some* instruments as input to the model, and generating in return the musical audio of *some other* instruments to accompany and to complement the user input, completing the song together. Such a model can potentially contribute to human-AI co-creation in songwriting [16] and enable new applications.

In technical terms, our work enhances the controllability of the model by allowing its generation to be steered on a user-provided audio track. It can be viewed as an interesting sequence-to-sequence problem where the model creates a "target sequence" of music that is to be played along to the input "source sequence." Besides requirement on audio quality, the coordination between the source and target sequences in terms of musical aspects such as style, rhythm, and harmony is also of central importance.

We note that, for controllability and the intelligibility of the generated singing, both Jukebox [7] and KaraSinger [8] have a lyrics encoder that allows their generation to be steered on textual lyrics. While being technically similar,



© Y.-K. Wu, C.-Y. Chiu, and Y.-H. Yang. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Y.-K. Wu, C.-Y. Chiu, and Y.-H. Yang, "JukeDrummer: Conditional Beat-aware Audio-Domain Drum Accompaniment Generation via Transformer VQ-VAE", in *Proc. of the 23rd Int. Society for Music Information Retrieval Conf.,* Bengaluru, India, 2022.

our *accompaniment generation* task ("audio-to-audio") is different from the lyric-conditioned generation task ("text-to-audio") in that the latter does not need to deal with the coordination between two audio recordings.

Specifically, we consider a *drum accompaniment generation* problem in our implementation, using a "drumless" recording as the input and generating as the output a drum track that involves the use of an entire drum kit. We use this as an example task to investigate the audio-domain accompaniment generation problem out of the following reasons. First, datasets used in musical source separation [17] usually consist of an isolated drum stem along with stems corresponding to other instruments. We can therefore easily merge the other stems to create paired data of drumless tracks and drum tracks as training data of our model. (In musical terms, drumless, or "Minus Drums" songs are recordings where the drum part has been taken out, which corresponds nicely to our scenario.) Second, we suppose a drum accompaniment generation model can easily find applications in songwriting [18], as it allows a user (who may not be familiar with drum playing or beat making) to focus on the other non-drum tracks. Third, audio-domain drum accompaniment generation poses interesting challenges as the model needs to determine not only the *drum patterns* but also the *drum sounds* that are supposed to be, respectively, rhythmically and stylistically consistent with the input. Moreover, the generated drum track is expected to follow a steady tempo, which is a basic requirement for a human drummer. We call our model the "JukeDrummer."

As depicted in Figure 1, the proposed model architecture contains an "audio encoder" (instead of the original text encoder [7, 8]) named the *drumless VQ encoder* that takes a drum-free audio as input. Besides, we experiment with different ways to capitalize an audio-domain beat and downbeat tracking model proposed recently [19] in a novel *beat-aware module* that extracts *beat-related information* from the input audio, so that the language model for generation (i.e., the Transformer) is better informed of the rhythmic properties of the input. The specific model [19] was trained on drumless recordings as well, befitting our task. We extract features from different levels, including low-level tracker embeddings, mid-level activation peaks, and high-level beat/downbeat positions, and investigate which one benefits the generation model the most.

Our contribution is four-fold. First, to our best knowledge, this work represents the first attempt to drum accompaniment generation of a full drum kit given drum-free mixed audio. Second, we develop a light-weight audio-to-audio Jukebox variant that takes an input audio of up to 24 seconds as conditioning and generates accompanying music in the domain of Mel-spectrograms (Section 3). Third, we experiment with different beat-related conditions in the context of audio generation (Section 4). Finally, we report objective and subjective evaluations demonstrating the effectiveness of the proposed model (Sections 6 & 7). [1] .

[1] We share our code and checkpoint at: `https://github.com/legoodmanner/jukedrummer`. Moreover, we provide audio examples at the following demo page: `https://legoodmanner.github.io/jukedrummer-demo/`

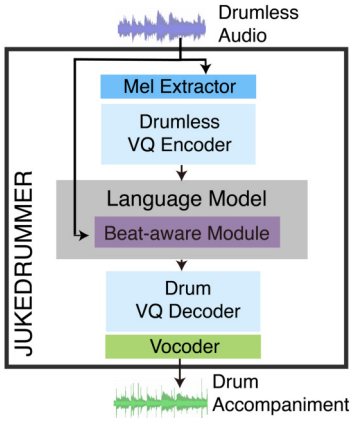


Figure 1: Diagram of the proposed JukeDrummer model for the inference stage. The training stage involves learning additional Drum VQ Encoder and Drumless VQ Decoder (see Figure 2) that are not used at inference time.

2. BACKGROUND

2.1 Related Work on Drum Generation

Conditional drum accompaniment generation has been studied in the literature, but only in the symbolic domain [20, 21], to the best of our knowledge. Dahale *et al.* [20] used a Transformer encoder to generate an accompanying symbolic drum pattern of 12 bars given a four-track, melodic MIDI passage. Makris *et al.* [21] adopted instead a sequence-to-sequence architecture with a bi-directional long short-term memory (BLSTM) encoder extracting information from the melodic input and a Transformer decoder generating the drum track for up to 16 bars in MIDI format. While symbolic-domain music generation has its own challenges, it differs greatly from the audio-domain counterpart studied in this paper, for it is not about generating sounds that can be readily listened to by human.

Related tasks that have been attempted in the literature with deep learning include symbolic-domain generation of a monophonic drum track (i.e., kick drum only) of multiple bars [4], symbolic-domain drum pattern generation [22–25], symbolic-domain drum track generation as part of a multi-track MIDI [26–29], audio-domain one-shot drum hit generation [30–34], audio-domain generation of drum sounds of an entire drum kit of a single bar [35], and audio-domain drum loop generation [36]. Jukebox [7] generates a mixture of sounds that include drums, but not an isolated drum track. By design, Jukebox does not take any input audio as a condition and generate accompaniments.

2.2 The Original Jukebox model

The main architecture of Jukebox [7] is composed of two components: a multi-scale vector-quantized variational autoencoder (VQ-VAE) [37–41] and an autoregressive Transformer decoder [14, 15]. The **VQ-VAE** is for converting a continuous-valued raw audio waveform into a sequence of so-called discrete *VQ codes*, while the **Transformer** establishes a language model (LM) of the VQ codes capable of generating new code sequences.

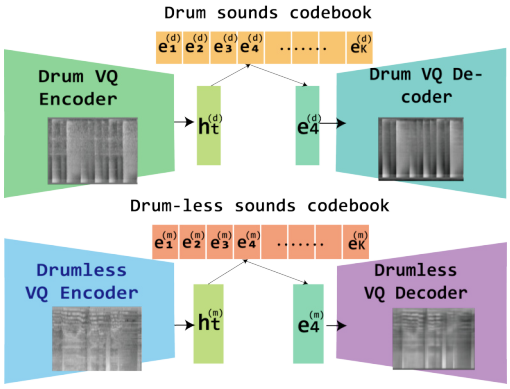


Figure 2: We use separate VQ-VAEs for the drumless and drum tracks, both operating on the Mel-spectrograms.

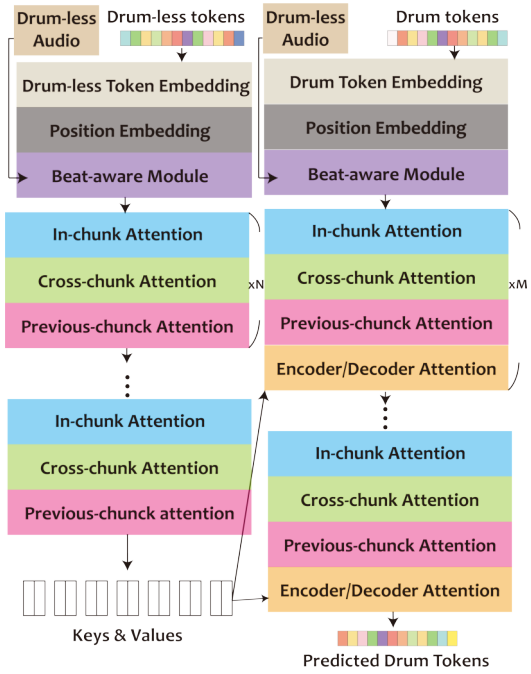


Figure 3: Details of our Transformer encoder/decoder.

Specifically, VQ-VAE consists of an encoder and a decoder, referred to as the VQ-VAE encoder $\mathcal{E}$ and VQ-VAE decoder $\mathcal{D}$ below, respectively. Given an audio waveform $\mathbf{x} \in \mathbb{R}^{1\times N}$, where N denotes the number of audio samples (e.g., N = 1,058,400 for a 24-second audio clip sampled at 44.1 kHz), the VQ-VAE encoder would convert $\mathbf{x}$ into a sequence of latent vectors $\{\mathbf{h}_t \in \mathbb{R}^D, t = 1, ..., T\}$, where the sequence length T is proportional to N, and the VQ-VAE decoder would have to reconstruct $\mathbf{x}$ from the "vector-quantized" version of the latent vectors, denoting as $\{\mathbf{h}'_t \in \mathbb{R}^D, t = 1, ..., T\}$, that is obtained by finding the nearest prototype vector $\mathbf{h}'_t = \mathbf{e}_z$ of each $\mathbf{h}_t$ in a codebook of prototype vectors $\{\mathbf{e}_k \in \mathbb{R}^D, k = 1, ..., K\}$, where K is the size of the codebook. Each prototype vector can be regarded as a cluster centroid in the latent space as a result of K-means clustering. The VQVAE encoder/decoder and the codebook are jointed learned by minimizing the reconstruction loss $\|\mathbf{x}_t - \mathcal{D}(\mathbf{h}'_t)\|_2^2$, and the commitment loss of the clustering $\|\mathcal{E}(\mathbf{x}_t) - sg(\mathbf{e}_z)\|_2^2$, where $\mathbf{x}_t$ denotes a slice of $\mathbf{x}$, $sg(.)$ the stop-gradient operation, and $\mathbf{h}_t = \mathcal{E}(\mathbf{x}_t)$.

Once the VQ-VAE is trained, the $\mathbf{x}$ can be viewed as a sequence of "IDs" $\{z_t \in \mathbb{Z}_{1:K}, t = 1, ..., T\}$, each corresponding to the index of the element of the codebook that is used to represent each slice of $\mathbf{x}$. The Transformer can then be trained on such sequences of IDs to learn the underlying language, or "composition rules," of the audio codes. Once trained, the transformer can be used to generate a novel sequence of codes, which can then be converted into an audio waveform by the VQVAE decoder.

As the waveforms are extremely long, Jukebox actually uses a "multi-scale" VQ-VAE that converts a waveform into three levels of codes, and accordingly three Transformers for building the LM at each level [7]. KaraSinger works on Mel-spectrograms but also uses multi-scale VQ-VAE [8]. We simplify their architecture by working on Mel-spectrograms with only one level of codes instead of multiple levels, as introduced below.

3. PROPOSED METHODS

In our task, we are given pairs of audio waveforms, namely a drumless stem $\mathbf{x}^{\mathrm{m}}$ and a drum stem $\mathbf{x}^{\mathrm{d}}$. Our goal is to train a model that can generate $\mathbf{x}^{\mathrm{d}}_*$ given an unseen $\mathbf{x}^{\mathrm{m}}_*$ from the test set. To achieve so, we propose several extensions of the Jukebox model. While we focus on drum accompaniment generation only, the same methodology may apply equally well to other conditional generation tasks.

Two VQ-VAEs. As illustrated in Figure 2, we build separate VQ-VAEs for $\mathbf{x}^{\mathrm{m}}$ and $\mathbf{x}^{\mathrm{d}}$ using drumless stems and drum stems respectively. Once trained, the drumless VQ-VAE encoder and the drum VQ-VAE encoder would convert $\mathbf{x}^{\mathrm{m}}$ and $\mathbf{x}^{\mathrm{d}}$ into $\{z^{\mathrm{m}}_t \in \mathbb{Z}_{1:K^{\mathrm{m}}}, t = 1, ..., T\}$ and $\{z^{\mathrm{d}}_t \in \mathbb{Z}_{1:K^{\mathrm{d}}}, t = 1, ..., T\}$ separately. Assuming that the drumless tracks are more diverse, we suggest use a larger codebook size for the drumless sounds codebook than the drum sounds codebook, namely $K^{\mathrm{m}} \geq K^{\mathrm{d}}$.

Mel VQ-VAE & Vocoder. To reduce computational cost, we build VQ-VAEs that take Mel-spectrograms as the input and target output. Specifically, we use the convolutional blocks of UNAGAN [5] to build a pair of VQ-VAE encoder and VQ-VAE decoder that has symmetric architectures (one downsampling and the other upsampling). We omit the details of UNAGAN due to space limit. Our Transformer accordingly learns the LM for codes of the Mel-spectrograms. Once a novel sequence of (drum) codes is generated by the Transformer, we use our (drum) VQ-VAE decoder to convert the codes into a Mel-spectrogram, and then use a neural *vocoder* [42] to convert the Mel-spectrogram into the corresponding waveform. While the latent vector $\mathbf{h}_t$ (and accordingly z_t) corresponds to a slice of waveform in the original Jukebox, here the latent vectors $\mathbf{h}^{\mathrm{m}}_t$ and $\mathbf{h}^{\mathrm{d}}_t$ (and z^{m}_t, z^{d}_t) both correspond to a fixed number of L frames of the short-time Fourier Transform (STFT) while computing the Mel-spectrograms.

Seq2seq Transformer. While the Jukebox model uses a Transformer decoder to model the sequences $\{z_t, t = 1, ..., T\}$ corresponding to mixtures of sounds, in our case we need a dedicated Transformer encoder to take the se-

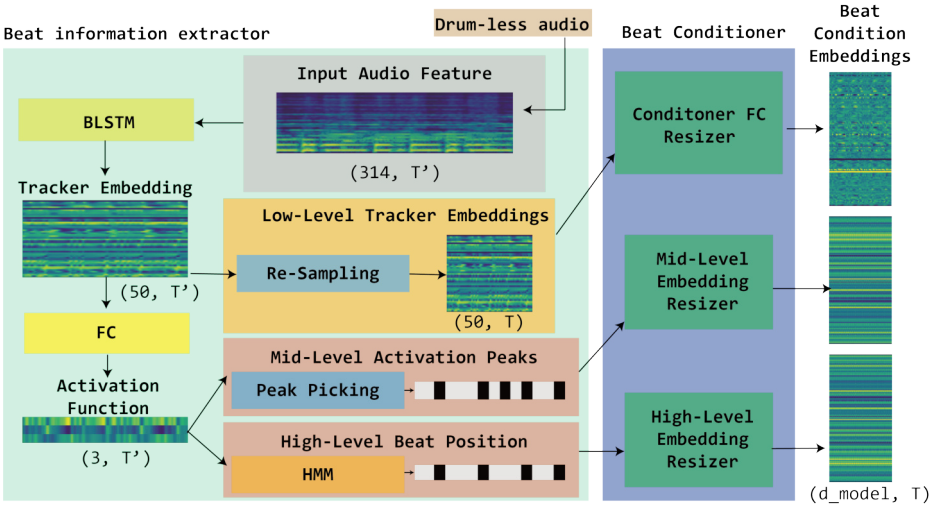


Figure 4: Details of the proposed beat-aware module that extracts beat condition embeddings from the drumless audio.

quence $\{z_t^{\mathrm{m}}, t = 1, ..., T\}$ corresponding to a drumless audio as the input, and a Transformer decoder to generate the sequence $\{z_t^{\mathrm{d}}, t = 1, ..., T\}$ corresponding to the accompanying drum audio as the output, leading to a sequence-to-sequence (seq2seq) architecture. Following Jukebox, we use the attention mechanism of the Scalable Transformer [15] in our Transformer encoder/decoder. As depicted in Figure 3, to determine its output z_i^{d} at each i, our Transformer decoder uses factorized "in-chunk," "cross-chunk" and "previous-chunk" attention layers to attend to the drum codes it generates previously, namely $z_{<i}^{\mathrm{d}} = \{z_t^{\mathrm{d}}, t = 1, ..., i-1\}$, to maintain the coherence of its output. A "chunk" here is a slice of the corresponding sequence. Moreover, the Transformer decoder uses "encoder/decoder attention" to attend to the final layer of the Transformer encoder to get contextual information from the drumless code sequence $\{z_t^{\mathrm{m}}, t = 1, ..., T\}$. To better synchronize the input and output, position embeddings are used. We refer readers to [7, 15] for details.

Beat-Aware Module. Figure 3 also shows that we use a novel "beat-aware module" (after the position embedding) in both Transformer encoder and decoder. We note that the code z_t^{m} is trained to provide essential information needed to reconstruct the drumless audio $\mathbf{x}_t^{\mathrm{m}}$, so the information carried by z_t might be *mixed*, covering different musical aspects including timbre, style, rhythm, etc. To supply the Transformer with *clear* rhythm-related information of $\mathbf{x}^{\mathrm{m}}$, we might need such a dedicated beat-aware module.

As depicted in Figure 4, the beat-aware module consists two sub-modules, the "beat information extractor" and the "beat conditioner." The former extracts beat-related information per frame from $\mathbf{x}^{\mathrm{m}}$ using an existing beat/downbeat tracker [19], while the latter incorporates that beat-related information for each t as a beat condition embedding $\mathbf{c}_t^{\mathrm{m}} \in \mathbb{R}^{d_{\mathrm{model}}}$ that has the same length d_{model} as the token embedding of the Transformers, and adds together the beat condition embedding and token embedding per t to serve as the input to the subsequent attention layers.

Despite that deep learning-based beat/downbeat trackers such as those proposed by Böck *et al.* [43–45] have achieved excellent performance for mainstream pop, rock or dance music, their performance and behaviors are influenced by the sound source composition (i.e., drum/non-drum sounds) of their training data [19, 46]. Considering that our input music is drumless, which is quite different from the music that existing common beat/downbeat trackers [44, 45] are trained with and trained for, we use in our beat information extractor the "non-drum tracker" developed by Chiu *et al.* [19] exclusively for drumless stems. In Section 4, we describe three ways to extract different levels of beat information feature from the non-drum tracker.

4. BEAT CONDITION EMBEDDING

As shown on the left of Figure 4, following the common design [44], the beat/downbeat tracker [19] uses BLSTM layers to get 50-dimensional "tracker embeddings" from $\mathbf{x}^{\mathrm{m}}$, and then uses fully-connected (FC) layers to compute 3-dimensional "activation functions" indicating the likelihood of observing a beat, downbeat, or non-beat at each frame. Finally, either a simple peak-picking or a hidden Markov model (HMM)-based algorithm [44] can be used to finalize the beat and downbeat time positions from the activation functions. We accordingly investigate extracting features from different stages of this processing pipeline.

Low-level (tracker) embeddings. We simply use the high-dimensional tracker embeddings, resampling it temporally, pooling them every L frames, and converting each of them to the desired length d_{model} with an FC.

High-level beat/downbeat positions. From the beat and downbeat time positions estimated by HMM, a frame can be labeled as either a beat, downbeat, or non-beat. We learn three d_{model}-dimensional embedding vectors corresponding to each case, and represent every L frames with one of the three vectors according to the frame labels.

Mid-level activation peaks. As the beat/downbeat positions are sparse over time, we consider a "denser" version by simply picking the peak positions of the activation func-

tions by `scipy.signal.find_peaks` [47] and similarly learning embedding vectors that are assigned to the frames according to whether a frame corresponds to a peak or not. Such a peak might represent a musical onset.

5. EXPERIMENT SETUP

Multi-track datasets for research on musical source separation [17] usually host recordings that each consists of an isolated drum stem along with stems corresponding to other instruments. We can simply take the drum stem as the target drum track, and the summation of the remaining stems as the input drumless track. In our implementation, we used the multi-track recordings from three datasets. MUSDB18 [17] contains 150 recordings, each of which has four stems corresponding to vocal, drums, bass, and "others." It is commonly used in source separation. MedleyDB [48] contains 196 multi-track recordings, each of which includes a drum stem along with many other stems. MixingSecret [49] has 257 multi-track recordings with various instruments, all including drums. We removed the 46 duplicate recordings between MUSDB18 and MedleyDB and the 100 duplicate recordings between MUSDB18 and MixingSerect, leading to 457 recordings to be used in our work. We randomly split the recordings into 80%, 10%, 10% as the training set, validation set (for parameter tuning), and testing set (for objective and subjective evaluation) at the "recording-level," ensuring that a recording does not appear in different splits.

All the recordings are sampled at 44.1 kHz and the stems are all monaural. Following Jukebox [7], we used 24-second audio clips in our work. We sliced each recording (and accordingly the stems) to 23.8-second audio clips with 50% temporal overlaps (as $2^{20}/44100 = 23.8$), and discarded the clips that do not contain any drum sounds. We then computed the Mel-spectrograms of each clip with PyTorch v1.7.1 with a Hann window of 1,024 samples for STFT, a hop size of 256 samples and 80 Mel-filter banks.

To evaluate the performance of the proposed JukeDrummer and validate the effectiveness of model components, we adopted the following variants in our experiments. [2]

- `seq2seq+beat (low)`: given a drumless clip $\mathbf{x}^{\mathrm{m}}$, we computed the drumless codes $\{z_t^{\mathrm{m}}\}$ and the low-level beat/downbeat tracker embeddings $\{\mathbf{c}_t^{\mathrm{m}}\}$ as the model input, to predict the drum codes $\{z_t^{\mathrm{d}}\}$ that eventually lead to the generated drum clip $\mathbf{x}^{\mathrm{d}}$, using the proposed seq2seq Transformer.
- `seq2seq+beat (mid)`: using the beat condition embeddings computed from the mid-level activation peaks (see Section 4) for $\{\mathbf{c}_t^{\mathrm{m}}\}$ instead.
- `seq2seq+beat (high)`: using the beat condition embeddings from the high-level beat/downbeat positions as $\{\mathbf{c}_t^{\mathrm{m}}\}$ instead.
- `seq2seq w/o beat`: to study the usefulness of the beat conditioning, we predicted $\{z_t^{\mathrm{d}}\}$ from $\{z_t^{\mathrm{m}}\}$, not using any beat-related conditions at all.
- `decoder+beat (low)`: to study the usefulness of the drumless codes $\{z_t^{\mathrm{m}}\}$, we used only the low-level beat condition embeddings $\{\mathbf{c}_t^{\mathrm{m}}\}$ as input to predict $\{z_t^{\mathrm{d}}\}$, via a Transformer decoder-only architecture (i.e., not seq2seq Transformer).
- `decoder w/o beat`: this is the baseline drummer that "plays its own," generating $\{z_t^{\mathrm{d}}\}$ autoregressively via a Transformer decoder without taking any information (neither $\{z_t^{\mathrm{m}}\}$ nor $\{\mathbf{c}_t^{\mathrm{m}}\}$) from the drumless clip $\mathbf{x}^{\mathrm{m}}$ it is supposed to play along to.

While the Mel-spectrogram for a clip in our case has 4,096 frames, the VQ-VAE encoder downsamples it to a sequence of $T = 1{,}024$ latent vectors, namely each corresponding to $L = 4$ frames. For VQ-VAE, we set the codebook size of drumless sounds $K^{\mathrm{m}} = 1{,}024$ and that of drums $K^{\mathrm{d}} = 32$ (so $K^{\mathrm{m}} \gg K^{\mathrm{d}}$), and the dimension of the latent vectors $D = 64$. The VQ-VAE encoders/decoders for both drumless and drums all have two layers. For those models employing a seq2seq LM, the Transformer encoder has 9 layers and the Transformer decoder has 20 layers. [3]

At inference time, we used the drum VQ decoder to convert the drum codes $\{z_t^{\mathrm{d}}\}$ to a Mel-spectrogram, which is then turned into the waveform of the drum clip $\mathbf{x}^{\mathrm{d}}$ by a HiFi-GAN V1 vocoder [42]. We trained the vocoder from scratch with audio of drum sounds from our dataset for 2.5 days, and then, inspired by [50, 51], fine-tuned it on the reconstructed Mel-spectrograms of the Drum VQ decoder.

6. OBJECTIVE EVALUATION

As audio-domain drum accompaniment generation is new, we propose customized metrics. Specifically, we use the "drum tracker" trained exclusively for drum stems by Chiu *et al.* [19], which follows exactly the same pipeline as the non-drum tracker (cf. Figure 4), to extract rhythmic features at three different levels for the ground-truth and generated drum sounds for the test split. We then compare the rhythmic features of the ground-truth and the generated in such three levels, using the mean square error for the low-level tracker embeddings (**TrackEmb-MSE**), cross entropy for the mid-level activation functions (**Act-Entropy**), and the F-measure for beat/downbeat estimation (**B/DB-F1**). The last one, computed by `mir_eval` [52], uses the beat positions of the ground-truth drums as the reference and those of the generated drums as the estimated beats. These metrics evaluates only the *rhythmic consistency* between the generated drums and the drumless audio (using its human-made drum track as a proxy), not other aspects such as stylistic consistency and audio quality, which will be evaluated subjectively in Section 7.

[2] A reviewer suggested that we should have compared our model with other accompaniment generation models that operate in the symbolic domain such as [20] and [21], saying that we can use existing audio samples or drum synthesis models to render their output to audio. However, the problem is that such a symbolic-domain accompaniment generation model also requires its input to be a MIDI file rather than audio.

[3] We used Adam as our optimizer with a learning rate of 0.0003 for both VQ-VAE and Transformer LM. We trained our LM with a batch size of 16, input feature dimension $d_{\text{model}} = 512$, two heads for multi-head attention, and the chunk size for factorized attention being 16. For those employing a decoder-only architecture, the number of layers of the Transformer decoder reduces to 15 because 5 of the layers corresponding to the "encoder/decoder attention" are removed.

Model	Objective metrics			Subjective MOS ($\in [1, 5]$)				
	TrackEmd-MSE↓	Act-Entropy↓	B/DB-F1↑	$\mathbf{R}^{m/d}$	$\mathbf{S}^{m/d}$	$\mathbf{Q}^{d}$	$\mathbf{R}^{d}$	$\mathbf{O}^{d}$
`seq2seq+beat (low)`	**.068**±.023	**.928**±.189	**.340**	**3.27**	**3.44**	**3.39**	**3.37**	**3.20**
`seq2seq+beat (mid)`	.077±.022	.963±.200	.212	—	—	—	—	—
`seq2seq+beat (high)`	.080±.024	.938±.160	.132	—	—	—	—	—
`seq2seq w/o beat`	.081±.022	.968±.233	.110	1.78	2.34	2.95	2.58	2.05
`decoder+beat (low)`	**.068**±.023	**.931**±.200	**.339**	**3.05**	**3.34**	**3.33**	**3.07**	**3.17**
`decoder w/o beat`	.087±.025	.987±.240	.114	1.59	1.83	2.56	1.88	1.73
Real data (not vocoded)	—	—	—	4.39	3.95	3.85	4.61	4.17

Table 1: Results of objective and subjective evaluation of variants of the proposed JukeDrummer model. The metrics are (from left to right): beat/downbeat tracking embedding MSE, beat/downbeat activation entropy, beat/downbeat F1, $\mathbf{R}^{m/d}$ (rhythmic consistency), $\mathbf{S}^{m/d}$ (stylistic consistency), $\mathbf{Q}^{d}$ (audio quality), $\mathbf{R}^{d}$ (rhythmic stability), $\mathbf{O}^{d}$ (overall). ↓/↑: the lower/higher the better; best two results (among the six model variants) per column highlighted in bold.

Result shown in the middle of Table 1 clearly shows that the models with low-level beat information achieve lower MSE, cross entropy, and higher F1, suggesting the usefulness of the low-level beat information for rhythmic consistency. The mid-level and high-level ones seem less effective (possibly because they are relatively monotonic), so we did not evaluate them further in Section 7. The objective scores also suggest that the Transformer encoder does not contribute much to rhythmic consistency.

7. SUBJECTIVE EVALUATION

With an online study, we solicited 22 anonymous volunteers to rate the result for 3 out of 15 random drumless tracks (each 23.8 seconds) from the test split. Each time, a volunteer listened to a drumless tracks ($\mathbf{x}_*^{m}$) first, and then (in random orders) the mixture (i.e., $\mathbf{x}_*^{m} + \mathbf{x}_*^{d}$) containing drum samples generated by four different models, plus the real human-made one (to set a high anchor). The volunteer then rated them in the following aspects on a 5-point Likert scale: **rhythmic consistency** between the drumless input and generated drums; **stylistic consistency** concerning the timbre and arrangement of the drumless input and generated drums; **audio quality** and **rhythmic stability** (whether the drummer follows a steady tempo) of the generated drum; and **overall** perceptual impression.

The mean opinion scores (MOS) in Table 1 show that `seq2seq+beat (low)` consistently outperforms the others, validating the effectiveness of using both the drumless codes and beat conditions. `decoder+beat (low)` performs consistently the second best, outperforming the two models without beat information significantly in three aspects according to paired t-test (p-value < 0.05), validating again the importance of the beat-aware module. Complementing Section 6, the MOS result suggests that the beat conditions seem more important than the drumless codes, though the best result is obtained with both.

Figure 5 further demonstrates that, given the same input, our model can generate multiple accompaniments with diversity in both beat and timbre. Diversity is an interesting aspect that is hard to evaluate, but it is desirable as there is no single golden drum accompaniment for a song. This may also explain why the F1 scores in Table 1 seem low.

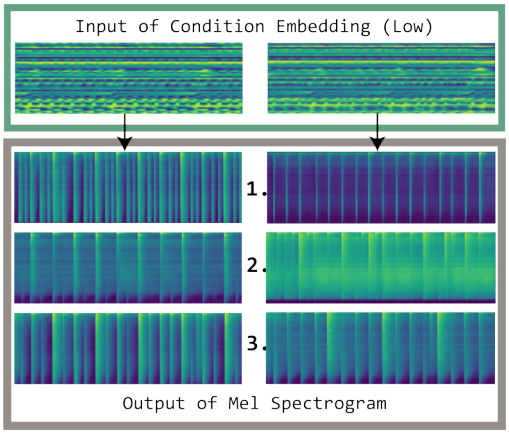


Figure 5: Two sets of three different generated samples by the same model given the same beat condition embedding.

Verbal feedbacks from the subjects confirm that our best model generates drum accompaniment that is rhythmically and stylistically consistent with the input, especially for band music or music with heavy use of bass. However, the model still has limits. At times the model generates total silence, though it can be avoided by sampling the LM again. The model may struggle to change its tempo going through different sections of a song. Moreover, the generation might be out-of-sync with the input in the beginning few seconds, until the model gets sufficient context. Please visit the demo page for various examples.

8. CONCLUSION

We have presented JukeDrummer, a novel audio-to-audio extension of OpenAI's JukeBox model capable of adding the drum part of a drumfree recording in the audio domain. To our knowledge, this represents the first attempt to audio-domain generation conditioned on drumless mixed audio. With objective and subjective evaluations, we validated the effectiveness of the customized VQ-VAE plus the seq2seq Transformer design, and the proposed beat-aware module. Among the beat conditions, we found that the low-level embeddings work the best. Future work can be done to further improve the language model (LM), and to extend our work to other audio-to-audio generation tasks.

9. ACKNOWLEDGEMENT

We are grateful to the anonymous reviewers for their valuable comments. This research is funded by grant NSTC 109-2628-E-001-002-MY2 from the National Science and Technology Council of Taiwan.

10. REFERENCES

[1] J. Engel, C. Resnick, A. Roberts, S. Dieleman, D. Eck, K. Simonyan, and M. Norouzi, "Neural audio synthesis of musical notes with WaveNet autoencoders," in *Proc. Int. Conf. Machine Learning*, 2017.

[2] J. Engel, K. K. Agrawal, S. Chen, I. Gulrajani, C. Donahue, and A. Roberts, "GANSynth: Adversarial neural audio synthesis," in *Proc. Int. Conf. Learning Representations*, 2019.

[3] C. Donahue, J. McAuley, and M. Puckette, "Adversarial audio synthesis," in *Proc. Int. Conf. Learning Representations*, 2019.

[4] S. Lattner and M. Grachten, "High-level control of drum track generation using learned patterns of rhythmic interaction," in *Proc. IEEE Work. Applications of Signal Processing to Audio and Acoustics*, 2019.

[5] J.-Y. Liu, Y.-H. Chen, Y.-C. Yeh, and Y.-H. Yang, "Unconditional audio generation with generative adversarial networks and cycle regularization," in *Proc. INTERSPEECH*, 2020.

[6] J. Engel, L. Hantrakul, C. Gu, and A. Roberts, "DDSP: Differentiable digital signal processing," in *Int. Conf. Learning Representations*, 2021.

[7] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," *arXiv preprint arXiv:2005.00341*, 2020.

[8] C.-F. Liao, J.-Y. Liu, and Y.-H. Yang, "KaraSinger: Score-free singing voice synthesis with VQ-VAE using Mel-spectrograms," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2022, pp. 956–960.

[9] C. M. Payne, "MuseNet," *OpenAI Blog*, 2019.

[10] C. Donahue *et al.*, "LakhNES: Improving multi-instrumental music generation with cross-domain pre-training," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2019.

[11] Y.-H. Chen, Y.-H. Huang, W.-Y. Hsiao, and Y.-H. Yang, "Automatic composition of guitar tabs by Transformers and groove modeling," in *Proc. Int. Soc. Music Inf. Retr. Conf.*, 2020.

[12] Z. Wang, Y. Zhang, Y. Zhang, J. Jiang, R. Yang, J. Zhao, and G. Xia, "PianoTree VAE: Structured representation learning for polyphonic music," in *Proc. Int. Soc. Music Inf. Retr. Conf.*, 2020.

[13] M. Suzuki, "Score Transformer: Transcribing quantized MIDI into comprehensive musical score," in *Proc. Int. Soc. Music Inf. Retr. Conf., Late-breaking demo paper*, 2021.

[14] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[15] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse Transformers," *arXiv preprint arXiv:1904.10509*, 2019.

[16] C. A. Huang, H. V. Koops, E. Newton-Rex, M. Dinculescu, and C. J. Cai, "AI song contest: Human-AI co-creation in songwriting," in *Proc. Int. Soc. Music Inf. Retr. Conf.*, 2020, p. 708–716.

[17] Z. Rafii *et al.*, "The MUSDB18 corpus for music separation," 2017. [Online]. Available: https://doi.org/10.5281/zenodo.1117372

[18] E. Deruty, M. Grachten, S. Lattner, J. Nistal, and C. Aouameur, "On the development and practice of AI technology for contemporary popular music production," *Transactions of the International Society for Music Information Retrieval*, vol. 5, no. 1, pp. 35–49, 2022.

[19] C.-Y. Chiu, W.-Y. Su, and Y.-H. Yang, "Drum-aware ensemble architecture for improved joint musical beat and downbeat tracking," *IEEE Signal Processing Letters*, vol. 28, pp. 1100–1104, 2021.

[20] R. Dahale, V. Talwadker, P. Verma, and P. Rao, "Neural drum accompaniment generation," in *Proc. Int. Soc. Music Inf. Retr. Conf., Late-breaking demo paper*, 2021.

[21] D. Makris, G. Zixun, M. Kaliakatsos-Papakostas, and D. Herremans, "Conditional drums generation using compound word representations," in *Proc. Artificial Intelligence in Music, Sound, Art and Design*, 2022, p. 179–194.

[22] J. Gillick, A. Roberts, J. Engel, D. Eck, and D. Bamman, "Learning to groove with inverse sequence transformations," in *Proc. Int. Conf. Machine Learning*, 2019.

[23] V. Thio, H.-M. Liu, Y.-C. Yeh, and Y.-H. Yang, "A minimal template for interactive web-based demonstrations of musical machine learning," in *Proc. Workshop on Intelligent Music Interfaces for Listening and Creation*, 2019.

[24] G. Alain, M. Chevalier-Boisvert, F. Osterrath, and R. Piche-Taillefer, "DeepDrummer: Generating drum loops using deep learning and a human in the loop," *arXiv preprint arXiv:2008.04391*, 2020.

[25] N. Tokui, "Can GAN originate new electronic dance music genres? – Generating novel rhythm patterns using GAN with genre ambiguity loss," *arXiv preprin: 2011.13062*, 2020.

[26] H.-W. Dong, W.-Y. Hsiao, L.-C. Yang, and Y.-H. Yang, "MuseGAN: Symbolic-domain music generation and accompaniment with multi-track sequential generative adversarial networks," in *Proc. AAAI Conf. Artificial Intelligence*, 2018.

[27] I. Simon, A. Roberts, C. Raffel, J. Engel, C. Hawthorne, and D. Eck, "Learning a latent space of multitrack measures," *arXiv preprint arXiv:1806.00195*, 2018.

[28] H.-M. Liu and Y.-H. Yang, "Lead sheet generation and arrangement by conditional generative adversarial network," in *Proc. IEEE. Conf. Machine Learning and Applications*, 2018, pp. 722–727.

[29] Y. Ren, J. He, X. Tan, T. Qin, Z. Zhao, and T.-Y. Liu, "PopMAG: Pop music accompaniment generation," in *Proc. ACM Multimedia Conf.*, 2020.

[30] J. Nistal, S. Lattner, and G. Richard, "DrumGAN: Synthesis of drum sounds with timbral feature conditioning using generative adversarial networks," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2020.

[31] A. Ramires, P. Chandna, X. Favory, E. Gómez, and X. Serra, "Neural percussive synthesis parameterised by high-level timbral features," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing*, 2020.

[32] C. Aouameur, P. Esling, and G. Hadjeres, "Neural drum machine: An interactive system for real-time synthesis of drum sounds," *arXiv preprint arXiv:1907.02637*, 2019.

[33] J. Drysdale, M. Tomczak, and J. Hockman, "Adversarial synthesis of drum sounds," in *Proc. Int. Conf. Digital Audio Effects*, 2020.

[34] ——, "Style-based drum synthesis with GAN inversion," in *Proc. Int. Soc. Music Inf. Retr. Conf., Late-breaking demo paper*, 2021.

[35] M. Tomczak, M. Goto, and J. Hockman, "Drum synthesis and rhythmic transformation with adversarial autoencoders," in *Proc. ACM Int. Conf. Multimedia*, 2020, p. 2427–2435.

[36] T. Hung, B. Chen, Y. Yeh, and Y. Yang, "A benchmarking initiative for audio-domain music generation using the Freesound Loop Dataset," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2021.

[37] A. v. d. Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Proc. Advances in Neural Information Processing Systems*, 2017.

[38] L. Kaiser *et al.*, "Fast decoding in sequence models using discrete latent variables," in *Proc. Int. Conf. Machine Learning*, 2018, pp. 2390–2399.

[39] A. Razavi *et al.*, "Generating diverse high-fidelity images with VQ-VAE-2," in *Proc. Advances in Neural Information Processing Systems*, 2019.

[40] A. Polyak *et al.*, "Speech resynthesis from discrete disentangled self-supervised representations," in *Proc. Interspeech*, 2021, pp. 3615–3619.

[41] J. Walker, A. Razavi, and A. v. d. Oord, "Predicting video with VQVAE," *arXiv preprint arXiv:2103.01950*, 2021.

[42] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," in *Proc. Advances in Neural Information Processing Systems*, 2020.

[43] S. Böck, F. Korzeniowski, J. Schlüter, F. Krebs, and G. Widmer, "madmom: A new Python audio and music signal processing library," *Proc. ACM Multimed. Conf.*, pp. 1174–1178, 2016.

[44] S. Böck *et al.*, "Joint beat and downbeat tracking with recurrent neural networks," in *Proc. Int. Soc. Music Information Retrieval Conf.*, 2016.

[45] S. Böck and M. E. P. Davies, "Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation," in *Proc. Int. Soc. Music Inf. Retr. Conf.*, 2020, p. 574–582.

[46] C.-Y. Chiu, J. Ching, W.-Y. Hsiao, Y.-H. Chen, W.-Y. Su, and Y.-H. Yang, "Source separation-based data augmentation for improved joint beat and downbeat tracking," in *Proc. Eur. Signal Process. Conf.*, 2021.

[47] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[48] R. Bittner *et al.*, "MedleyDB: A multitrack dataset for annotation-intensive MIR research," in *Proc. Int. Soc. Music Inf. Retr. Conf.*, 2014, [Online] http://medleydb.weebly.com/.

[49] "The 'Mixing Secrets' free multitrack download library," [Online] https://www.cambridge-mt.com/ms/mtk/.

[50] K.-W. Kim, S.-W. Park, J. Lee, and M.-C. Joe, "ASSEM-VC: Realistic voice conversion by assembling modern speech synthesis techniques," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2022.

[51] X. Tan *et al.*, "NaturalSpeech: End-to-end text to speech synthesis with human-level quality," *arXiv preprint arXiv:2205.04421*, 2022.

[52] C. Raffel *et al.*, "mir_eval: A transparent implementation of common MIR metrics," in *Proc. Int. Soc. Music Inf. Retr. Conf.*, 2014, pp. 367–372.